

Embodied Intelligence Club - Christmas Project

Humanoid Locomotion Simulator

Stathis Liapodimitris

1 Design Pattern Map

For this project, the goal was to build a locomotion simulator that is designed like a properly structured program. To make the code clean, modular, and easy to modify later, I implemented the mandatory design patterns that were suggested in the explanation of the project. Below is a breakdown of how I used each pattern and the justification for each one.

1. Strategy Pattern

Classes: `DynamicsModel` (`LIPMContinuousEuler`, `LIPMDiscreteLinearAB`), `ControlPolicy` (`CapturePointPolicy`, `LeastSquaresPolicy`).

Justification: This pattern lets me easily swap out the physics engine and the policy of the robot. Instead of hardcoding one specific way to do the math, the simulator just talks to a generic interface. This means I can switch between simple Euler integration and exact discrete math, or change the walking logic from a Capture Point strategy to a Least Squares approach, without having to rewrite any of the main simulation code.

2. Factory Method Pattern

Classes: `SimulatorFactory`.

Justification: Instead of manually creating my physics models and control policies inside the main script, I built a Factory class to handle it. The factory reads a configuration dictionary and builds the correct objects for me automatically. This way I never directly call constructors outside of the factory. It makes testing easier since I can run completely different scenarios just by editing a simple config dictionary.

3. Template Method Pattern

Classes: `BaseSimulator`, `ScenarioSimulator`.

Justification: The `BaseSimulator` defines a strict, step-by-step routine for the simulation loop inside the `step()` method: compute control, apply constraints, calculate physics, add pushes, and log data. I locked this sequence down so the core math order cannot be accidentally messed up. If I want to change how a specific part works (like I did when adding stepping functionality with a type of sample and hold logic), I use the `ScenarioSimulator` subclass to safely override just that specific hook.

4. Observer Pattern

Classes: `Observer` (Interface), `StateLoggerObserver`, `FallCounterObserver`, `BaseSimulator` (Subject).

Justification: I wanted to ensure the physics engine focuses only on doing math, not on printing text, saving arrays, or drawing graphs. The Observer pattern solves this. Every time the simulator finishes a step, it simply broadcasts the new state. My observers (like the logger or the referee that counts falls) listen for this broadcast and record what they need. This keeps the core simulation loop fast and minimal.